

ProjectF

Unity 기반 자동화·생존 시뮬레이션의 시스템 설계

PROJECT DEFINITION 자원 채집부터 생산 설비, 컨베이어 물류, 철도 운송, 유체 네트워크까지 하나의 월드에서 연결되는 1인 개발 자동화 게임입니다. 확장되는 공장의 처리량과 월드 상태의 정확성을 동시에 유지하는 것을 핵심 기술 목표로 삼았습니다.

기술 스냅샷

- 엔진: Unity 6000.4.0f1 · 렌더 파이프라인: URP 17.4
- 언어·런타임: C# · Burst 1.8.28 · Collections 6.4.0 · Mathematics 1.3.3
- 핵심 시스템: 컨베이어, 생산 설치물, 철도·차량, 파이프·유체, 청크 스트리밍, 세이브/로드
- 개발 방식: 성능 민감 경로를 측정 가능한 구조로 만들고 AI를 분석·구현 보조 도구로 활용

설계 원칙

- ① 논리 상태와 표현 계층을 분리합니다.
- ② 활성·비활성 청크의 상태를 하나의 권위 모델로 연결합니다.
- ③ 고비용 처리는 배치화하고 실패 시 안전한 대체 경로를 둡니다.
- ④ 프로파일러와 진단 지표로 최적화 결과를 검증할 수 있게 만듭니다.

한택근 | Client Programmer

ProjectF v0.1 · Repository-based technical brief

01 · SYSTEM ARCHITECTURE

확장 가능한 공장을 위한 계층형 구조

ProjectF 의 핵심은 '월드에서 존재하는 모든 객체를 항상 실행하지 않는다'는 판단입니다. 입력과 설치물 로직, 청크 수명주기, 영속 상태, 렌더링·진단을 분리하여 각 계층의 비용과 책임을 통제합니다.



청크 스트리밍과 상태 수명주기

- TerrainChunkStreamingScheduler 는 생성·언로드 요청을 큐와 HashSet 으로 중복 제거하고, 코루틴 예산에 맞춰 순차 처리합니다.
- 청크 언로드 시 설치물과 운반 아이템을 BlockStateStore 로 캡처하고, 재진입 시 지형 생성 후 제한된 개수만 단계적으로 복원합니다.
- 화면 밖 청크는 InstallationBackgroundSimulator 가 저장 상태를 기준으로 진행시켜, 비활성 영역도 생산 흐름이 단절되지 않게 합니다.

논리 상태와 표현의 분리

GameObject 의 존재 여부를 게임 상태의 진실 원본으로 삼지 않습니다. 활성 청크의 런타임 객체와 비활성 청크의 저장 상태가 동일한 식별자·연결 정보를 공유하고, 렌더러는 그 결과를 소비하는 표현 계층으로 동작합니다. 이 구조는 대규모 월드에서 객체 수를 줄이면서도 저장·복원 정합성을 유지하기 위한 기반입니다.

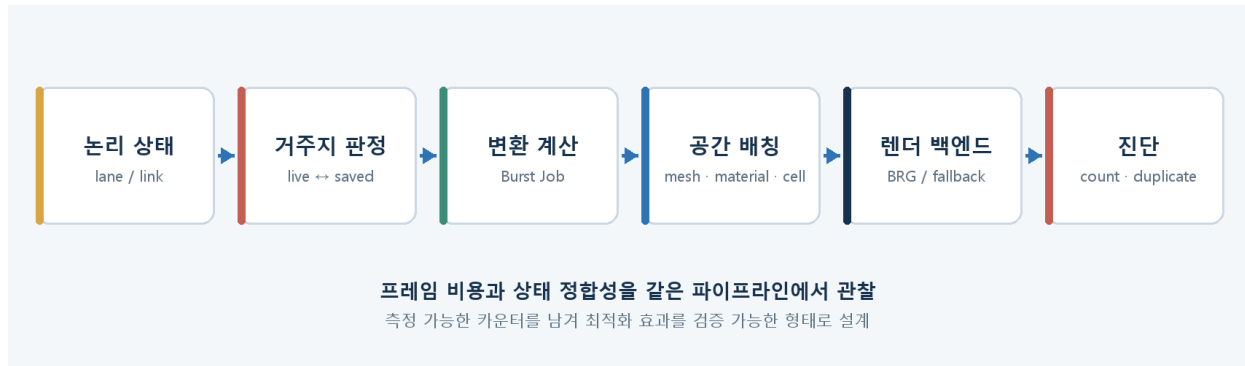
핵심 효과 청크 경계를 넘나드는 생산·운송 상태를 보존하면서, 프레임당 생성·복원 작업량을 제한할 수 있습니다.

CODE ANCHORS TerrainChunkStreamingScheduler.cs · TerrainGenerator.ChunkPersistence.cs · BlockStateStore.cs · InstallationBackgroundSimulator.cs

02 · CONVEYOR OPTIMIZATION

컨베이어를 ‘개별 오브젝트’가 아닌 데이터 흐름으로 처리

공장 규모가 커질수록 벨트와 운반 아이템은 CPU 갱신, Transform 계산, 드로우콜, GC 할당을 동시에 압박합니다. 이를 해결하기 위해 논리 상태 → 변환 계산 → 공간 배치 → 렌더링 → 진단으로 이어지는 파이프라인을 구성했습니다.



병목	구현 방식	검증 지점
CPU·GC	Persistent NativeArray 와 Burst IJobParallelFor 로 아이템 변환을 일괄 계산	작업 수·프레임 시간·할당
드로우콜	메시·머티리얼·공간 셀 기준 배치, BatchRendererGroup 사용	등록 벨트·예상 드로우콜
호환성	BRG 지원 여부를 확인하고 RenderMeshInstanced 로 안전하게 풀백	백엔드 실패·전환 로그
상태 오류	활성·저장 아이템을 함께 집계하고 중복 거주·잘못된 링크 탐지	live/saved/duplicate 카운터

최적화의 기준

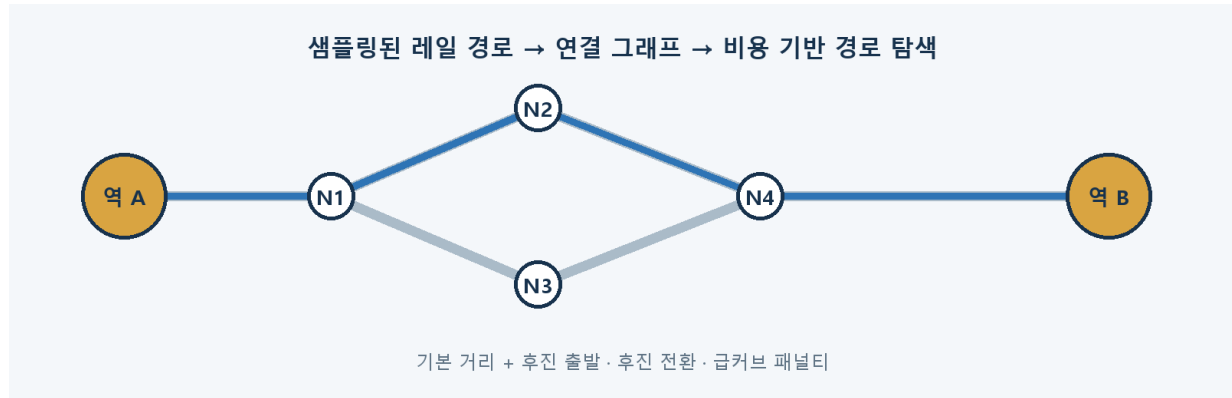
작은 작업은 메인 스레드에서 처리해 잡 스케줄링 오버헤드를 피하고, 임계값을 넘으면 Burst 병렬 경로로 전환합니다. 버퍼 용량은 2 의 거듭제곱으로 확장해 재할당 빈도를 낮추고, 렌더 백엔드는 기능 지원과 셰이더 호환성을 확인한 뒤 선택합니다.

검증 원칙 현재 저장소에는 정량 벤치마크 결과 대신 프로파일러·진단 카운터가 준비되어 있습니다. 따라서 성능 향상 수치를 과장하지 않고, 같은 맵·같은 아이템 수에서 반복 측정할 수 있는 구조적 기반을 성과로 제시합니다.

03 · NETWORK SIMULATION

철도 자동운전: 곡선을 그래프로 바꾸고 비용으로 판단

레일은 단순한 직선 연결이 아니라 샘플링된 곡선과 분기점의 집합입니다. 각 레일의 렌더 경로를 노드·엣지 그래프로 변환하고, 역을 출발지와 목적지로 연결해 비용 기반 경로를 탐색합니다.



경로 계획과 주행 상태

- AutoDriveRoutePlanner 는 그래프 버전을 캐시하고, 레일 연결 변경 시에만 무효화하여 불필요한 재구성을 줄입니다.
- 거리뿐 아니라 후진 출발, 진행 방향 전환, 급격한 회전에 패널티를 부여해 실제 주행에 가까운 경로를 선택합니다.
- Idle → Planning → Moving → Docking → WaitingAtStation/Arrived 상태를 분리하고, 연료·경로·선로 점유 대기 원인을 별도 상태로 노출합니다.
- 열차 연결 그래프를 순회해 편성 전체를 구성하고, 경로 세그먼트·커서·look-ahead 를 저장해 주행을 이어갑니다.

같은 원칙을 유체 네트워크에 적용

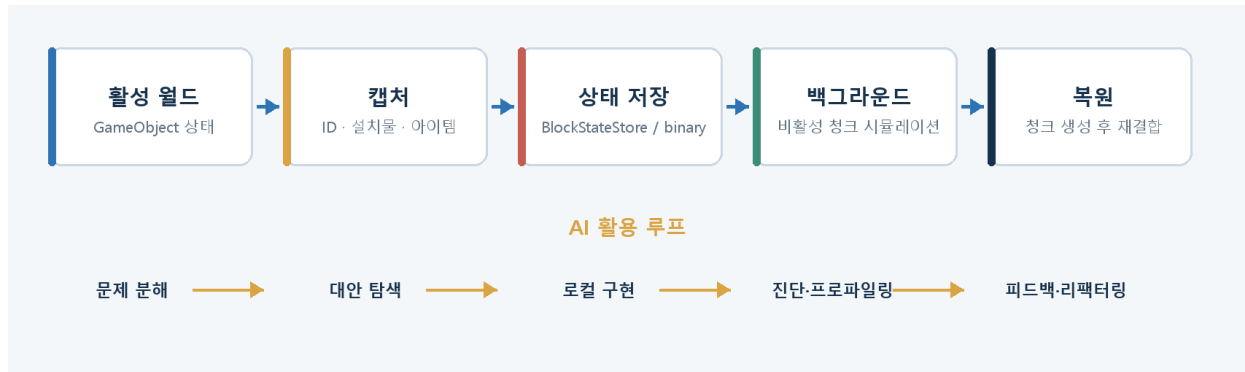
Pipe·UndergroundPipe·FluidTank·Pump 는 연결 토폴로지를 BFS 로 탐색하고 버전 단위로 캐시합니다. 지하 파이프의 원격 링크까지 같은 네트워크로 묶어 유체량을 평준화하되, 연결이 바뀔 때만 네트워크를 다시 계산합니다. 철도와 유체는 모두 '연결 관계를 그래프로 명시하고 변경 시점에만 갱신한다'는 공통 설계를 사용합니다.

CODE ANCHORS SteamTrain.cs · Train.cs · Railroad.cs · RailConnectionUtility.cs · Pipe.cs · UndergroundPipe.cs

04 · PERSISTENCE, VALIDATION & AI

세이브·복원과 AI 활용을 하나의 검증 루프로

세이브 시스템은 설치물 타입별 상태, 위치·회전, 내부 아이템, 연결 정보, 열차 자동운전 상태를 바이너리 포맷으로 직렬화합니다. 체크 저장 상태와 전체 세이브가 동일한 복원 규칙을 공유하도록 하여, 스트리밍과 재접속에서 다른 결과가 생기지 않게 설계했습니다.



AI 를 복잡한 구현의 가속기로 사용

Factorio 의 컨베이어 최적화 방식을 공부한 뒤, AI 와 함께 오브젝트 단위 갱신의 병목을 데이터 중심 파이프라인으로 재구성했습니다. AI 는 대형 코드베이스 탐색, BRG-Burst 처럼 익숙하지 않은 API 의 대안 비교, 반복되는 로직의 리팩터링 초안에 활용했습니다. 저는 상태의 진실 원본, 폴백 조건, 성능 임계값을 정의하고 생성된 코드를 직접 검토했으며, 진단 결과를 다시 입력해 중복·예외 분기를 정리했습니다. 이를 통해 혼자서는 진입 비용이 컸던 렌더 배칭과 병렬 변환 로직을 실제 프로젝트 구조 안에 구현할 수 있었습니다.

검증 도구와 다음 단계

- MapObjectProfilerTool: TCP 로 FPS-프레임 시간·틱 카운터를 조회하는 독립 프로파일러
- ConveyorRuntimeDiagnostics: 생성/저장 아이템 수, 중복 거주, 레인·링크 오류를 한 화면에서 확인
- BeltAlignmentProbe: 에디터에서 벨트 정렬과 샘플 위치를 재현하는 진단 진입점
- 다음 단계: 고정 맵 기반 회귀 벤치마크, 저장 포맷 테스트, namespace-asmdef 단위의 점진적 책임 분리

기술적 결론 ProjectF 는 단순 기능 나열이 아니라, 대규모 자동화 월드에서 상태 정합성·성능·디버깅 가능성을 함께 다루는 시스템 프로젝트입니다.